



## Cấu trúc dữ liệu: Set, Map, Multiset

## Nội dung

- 1. Tại sao cần set và map?
- 2. std::set — tập hợp các giá trị khác nhau, đã sắp xếp
  - Khai báo và thao tác cơ bản
  - Tìm phần tử lớn nhất / nhỏ nhất
  - lower\_bound và upper\_bound — siêu mạnh!
  - Ví dụ: Tìm số lớn nhất nhỏ hơn  $x$
- 3. std::multiset — cho phép trùng
- 4. std::map — ánh xạ key  $\rightarrow$  value
  - Đếm tần suất (cực kỳ phổ biến!)
- 5. unordered\_set / unordered\_map — bảng băm (hash)
- 6. Mẫu bài toán điển hình
  - Mẫu 1: Số khác nhau trong mảng
  - Mẫu 2: Có cặp  $(i, j)$  với  $a_i + a_j = T$  không?
  - Mẫu 3: Online — sau mỗi lần thêm 1 số, hỏi số nhỏ thứ  $k$
  - Mẫu 4: Đếm nghịch thế (số cặp  $(i < j)$  với  $a_i > a_j$ )
  - Mẫu 5: Hai cấu trúc song song — duy trì cả phần tử và "khoảng cách" giữa các phần tử
- 7. Bẫy thường gặp & Mẹo
- 8. Tổng kết

Sau khi đã làm quen với prefix sum 2D ở bài 2, đã đến lúc mở rộng kho vũ khí của bạn với các **cấu trúc dữ liệu cao cấp hơn** — `set`, `map`, và `multiset`.

Đây là những công cụ **cực kỳ mạnh mẽ**: chúng tự động giữ phần tử theo thứ tự, thêm/xóa/tìm kiếm trong  $O(\log n)$ , và giúp bạn giải nhiều bài toán mà nếu chỉ dùng mảng sẽ rất khó.

## 1. Tại sao cần set và map ?

Hãy xem bài toán: **Cho dãy  $n$  số. Với mỗi truy vấn, hỏi xem một số  $x$  có trong dãy không.**



Tiếng Việt



$q = 10^6$  truy vấn  $\rightarrow 10^6$  phép tính  $\rightarrow TLE$ :

- **Cách dùng `set`**: Đặt tất cả số vào `set`, kiểm tra trong  $O(\log n)$  — tổng  $O((n + q) \log n) \approx 2 \times 10^6 \rightarrow AC!$

Đó là một trong rất nhiều ví dụ. `set` và `map` giúp:

- Kiểm tra phần tử có tồn tại không
- Đếm số phần tử khác nhau
- Tìm phần tử lớn nhất/nhỏ nhất hơn  $x$
- Lưu key-value mà key tự động được sắp xếp

## 2. `std::set` — tập hợp các giá trị khác nhau, đã sắp xếp

`set` lưu các phần tử **khác nhau từng đôi**, tự động sắp xếp tăng dần.

“

**Lưu ý cho người dùng Python:** Python **không có** cấu trúc tương đương

`std::set` / `std::map` đã sắp xếp trong thư viện chuẩn. `set` / `dict` của Python là **bảng băm** —  $O(1)$  trung bình nhưng **không có thứ tự**, không có `Lower_bound`. Đối với các bài cần tập/map có thứ tự (đây là lý do chính của bài học này), **hãy dùng C++**. Toàn bộ ví dụ trong bài này được viết bằng C++.

## Khai báo và thao tác cơ bản

C++



```

set<int> s;

s.insert(5);           // O(log n)
s.insert(3);
s.insert(5);           // Không thêm nữa, vì 5 đã có
s.erase(3);           // Xóa giá trị 3, O(log n)

if (s.count(5)) {}     // Kiểm tra có tồn tại không, O(log n)

cout << s.size();      // Số phần tử

for (int x : s) cout << x << ' '; // Duyệt theo thứ tự tăng dần

```

**Thời gian các thao tác chính:** insert, erase, find, count — đều  $O(\log n)$ .

**Tìm phần tử lớn nhất / nhỏ nhất**

C++

```

int mn = *s.begin();           // Nhỏ nhất
int mx = *s.rbegin();          // Lớn nhất (rbegin = reverse begin)

```

**lower\_bound** và **upper\_bound** — siêu mạnh!

- `s.lower_bound(x)` : trở tới phần tử **nhỏ nhất**  $\geq x$
- `s.upper_bound(x)` : trở tới phần tử **nhỏ nhất**  $> x$

C++

```

auto it = s.lower_bound(7);
if (it != s.end()) cout << *it; // Số nhỏ nhất  $\geq 7$ 

```

**Lưu ý:** Phải dùng `s.lower_bound(x)`, **không phải** `lower_bound(s.begin(), s.end(), x)` — cái sau là  $O(n)$  vì set không phải mảng truy cập ngẫu nhiên!

**Ví dụ: Tìm số lớn nhất nhỏ hơn  $x$**



Tiếng Việt



```
auto it = s.lower_bound(x);
if (it != s.begin()) {
    --it;
    cout << *it;    // Số lớn nhất < x
}
```

### 3. `std::multiset` — cho phép trùng

`multiset` giống `set` nhưng **cho phép phần tử trùng nhau**.

C++

```
multiset<int> ms;
ms.insert(5);
ms.insert(5);    // OK, có 2 phần tử 5
ms.insert(3);

cout << ms.count(5);    // 2
ms.erase(ms.find(5));    // Xóa MỘT phần tử 5 (quan trọng!)
// ms.erase(5) sẽ xóa TẤT CẢ phần tử 5 – cẩn thận!
```

**Bẫy thường gặp:** `ms.erase(value)` xóa **tất cả** phần tử bằng `value`. Để xóa **chỉ một**, dùng `ms.erase(ms.find(value))`.

### 4. `std::map` — ánh xạ key → value

`map` lưu các cặp `(key, value)`, key tự động sắp xếp.

C++



Tiếng Việt



```
map<string, int> cnt;

cnt["apple"] = 3;
cnt["banana"] = 5;
cnt["apple"]++;           // Giờ apple = 4

if (cnt.count("apple")) {} // Kiểm tra key tồn tại

for (auto [k, v] : cnt) { // C++17, duyệt theo thứ tự key
    cout << k << ' ' << v << '\n';
}
```

**Bấy:** `cnt[key]` mà `key` chưa tồn tại sẽ **tự động tạo** với giá trị mặc định (0 cho int). Nếu chỉ muốn kiểm tra, dùng `cnt.count(key)` hoặc `cnt.find(key)`.

### Đếm tần suất (cực kỳ phổ biến!)

C++

```
map<int, int> freq;
for (int x : a) freq[x]++;

cout << freq.size(); // Số khác nhau
int mx = 0;
for (auto [k, v] : freq) mx = max(mx, v);
```

## 5. `unordered_set` / `unordered_map` — bảng băm (hash)

- `set` / `map` : cây cân bằng (Red-Black Tree),  $O(\log n)$ , **có thứ tự**
- `unordered_set` / `unordered_map` : bảng băm,  $O(1)$  trung bình, **không có thứ tự**

C++



Tiếng Việt



```
unordered_map<int, int> cnt;
cnt[x]++; // O(1) trung bình
```

### Khi nào dùng cái nào?

- Cần **thứ tự** (lower\_bound, duyệt tăng dần) → `set` / `map`
- **Không cần thứ tự**, chỉ cần tra cứu nhanh → `unordered_set` / `unordered_map`

**Cảnh báo:** `unordered_map` có thể bị **anti-hash attack** (TLE do chủ ý). Trong CP an toàn nhất là dùng `map` trừ khi thực sự cần  $O(1)$ .

## 6. Mẫu bài toán điển hình

### Mẫu 1: Số khác nhau trong mảng

C++

```
set<int> s(a.begin(), a.end());
cout << s.size();
```

### Mẫu 2: Có cặp $(i, j)$ với $a_i + a_j = T$ không?

Duyệt  $i$ , kiểm tra  $T - a_i$  trong set:

C++

```
set<int> seen;
for (int x : a) {
    if (seen.count(T - x)) { /* tìm thấy */ }
    seen.insert(x);
}
```

### Mẫu 3: Online — sau mỗi lần thêm 1 số, hỏi số nhỏ thứ $k$

`policy_tree`).

Với  $k = 1$  (nhỏ nhất): chỉ cần `*ms.begin()`.

#### Mẫu 4: Đếm nghịch thế (số cặp $i < j$ với $a_i > a_j$ )

Bài toán này cần **đếm nhanh số phần tử lớn hơn**  $a_j$  trong tập đã thấy — `set` / `map` thường không hỗ trợ trực tiếp ( $O(n)$ ). Cần Fenwick tree hoặc Policy-based tree (`__gnu_pbds`) — sẽ học ở khóa **Vàng**.

#### Mẫu 5: Hai cấu trúc song song — duy trì cả phần tử và "khoảng cách" giữa các phần tử

Một số bài cần **không chỉ tập các giá trị**, mà còn **các khoảng trống/khoảng cách** giữa các giá trị liên kề. Mẹo: duy trì **hai cấu trúc song song**:

- `set<int> S` — chứa các giá trị hiện có (sắp xếp).
- `multiset<int> gaps` — chứa các khoảng cách giữa hai phần tử kề nhau trong  $S$ .

Khi thêm/xóa  $x$  vào  $S$ :

1. Tìm hai phần tử  $\ell, r$  kề  $x$  trong  $S$ .
2. Xóa khoảng cũ  $r - \ell$  khỏi `gaps`.
3. Thêm hai khoảng mới  $x - \ell$  và  $r - x$  vào `gaps`.
4. Cập nhật  $S$ .

Khoảng dài nhất hiện tại = `*gaps.rbegin()` trong  $O(1)$ .

C++

```
set<int> S = {0, L}; // hai biên
multiset<int> gaps = {L};

void addPoint(int x) {
    auto it = S.lower_bound(x);
    int r = *it, l = *prev(it);
    gaps.erase(gaps.find(r - l)); // xóa MỘT khoảng cũ
    gaps.insert(x - l);
    gaps.insert(r - x);
    S.insert(x);
}

int longestGap() { return *gaps.rbegin(); }
```



Tiếng Việt



lớn nhất. Bài luyện tập kinh điển là **CSES Traffic Lights** (`cses1163`).

## 7. Bẫy thường gặp & Mẹo

- `set` **không có random access**: không có `s[i]`. Phải dùng iterator hoặc `next(s.begin(), i)` — và cái sau là  $O(i)$ !
- Iterator bị invalid khi erase**: lưu lại trước hoặc dùng `it = s.erase(it)`.
- Hằng số lớn**: `set` / `map` chậm hơn array khoảng 5-10 lần. Nếu  $n$  lớn ( $\geq 10^6$ ) và chỉ cần kiểm tra tồn tại trong tập hợp số nhỏ, hãy dùng `vector<bool>` hoặc `bitset`.
- `map[key]` **tạo entry**: nếu chỉ tra cứu, dùng `find` hoặc `count`.

## 8. Tổng kết

| CẤU TRÚC                       | CHO PHÉP TRÙNG  | CÓ THỨ TỰ | THAO TÁC    |
|--------------------------------|-----------------|-----------|-------------|
| <code>set</code>               | Không           | Có        | $O(\log n)$ |
| <code>multiset</code>          | Có              | Có        | $O(\log n)$ |
| <code>map</code>               | Key không trùng | Có        | $O(\log n)$ |
| <code>unordered_set/map</code> | tùy             | Không     | $O(1)$ TB   |

### Khi nào dùng `set` / `map` ?

- Cần kiểm tra tồn tại nhanh
- Cần `lower_bound` / `upper_bound` (tìm "gần nhất")
- Cần phần tử min/max trong tập đang biến đổi (`insert/erase`)
- Đếm tần suất với key bất kỳ (`string`, `pair`, ...)

Hãy luyện thật nhiều bài tập — `set` / `map` sẽ trở thành công cụ mặc định trong tay bạn!

## Bài tập



BASIC SET

0 / 10



Đếm lũy thừa

0 / 10





Tiếng Việt



|                                          |        |
|------------------------------------------|--------|
| Xóa k phần tử                            | 0 / 15 |
| Sắp xếp theo tần suất                    | 0 / 15 |
| minict26                                 | 0 / 15 |
| Bài 2. Cân bằng (HSG 9 Hà Nội 2025-2026) | 0 / 15 |
| Kết nối (DUTPC'21)                       | 0 / 15 |
| CSES - Concert Tickets   Vé hòa nhạc     | 0 / 25 |
| CSES - Traffic Lights   Đèn giao thông   | 0 / 30 |

proudly powered by **DMOJ** | [developed by LQDJudge team](#) Tiếng Việt (vi) ▼